AquaLicious

System Algorithms, Mathematical Formulas & Concurrency Controls

Architecture: React + Express + PostgreSQL
Timezone: Asia/Manila (UTC+8)
Generated: November 12, 2026

1. Sales Analytics & Trend Forecasting

Source: `backend/src/services/reportService.ts`

7-Day Simple Moving Average (SMA)

Computes the average daily revenue over the last **7 completed business days** (`today - 7` through `today - 1` in `Asia/Manila`). Today's partial revenue is excluded so unfinished day totals do not create artificial downward bias.

$$\text{SMA}_7 = \left(\Sigma_{i=1..7} \text{Revenue}_{\text{day } i} \right) / 7$$

Ordinary Least Squares (OLS) Linear Regression

Fits a least-squares trend line across the 7 completed days where `x = 1..7` (day index) and `y = daily revenue` to measure the daily peso rate of change (slope):

$$\text{Slope} = [\, n(\Sigma xy) - (\Sigma x)(\Sigma y) \,] \,/\, [\, n(\Sigma x^2) - (\Sigma x)^2 \,] \quad (\text{where } n = 7)$$

Percentage Trend Classification

Normalizes the daily peso slope by the 7-day mean revenue (`meanY`) when `meanY > 0` :

$$\text{Percentage Slope} = \left(\text{Slope} \,/\, \text{Mean Daily Revenue} \right) \times 100\%$$

Condition	Trend Label	Interpretation
`percentageSlope ≥ +5%`	**Growing**	Upward sales trajectory over the past 7 completed days
`-5% < percentageSlope < +5%`	**Stable**	Consistent daily revenue within normal variance
`percentageSlope ≤ -5%`	**Declining**	Downward sales trajectory over the past 7 completed days

Historical Cost Snapshot Gross Profit

Calculates profit per completed order line using the unit cost frozen at the time of checkout (`unitCostSnapshot`), protecting historical reports from future cost updates:

$$\text{Gross Profit} = \Sigma \left(\text{unitPrice} - \text{unitCostSnapshot} \right) \times \text{quantity}$$

2. Inventory & Replenishment (ROP)

Source: `backend/src/services/inventoryService.ts` & `frontend/src/components/RopCalculatorModal.tsx`

Average Daily Usage (ADU)

Aggregates total units sold per product from `COMPLETED` and `CONFIRMED` orders over the previous 7 completed calendar days (`subDays(now, 7)` to `subDays(now, 1)`):

$$\text{ADU} = \text{Total Units Sold (7-Day Window)} \,/\, 7$$

Reorder Point (ROP) Formula

Determines the minimum stock threshold that covers expected demand during supplier delivery lead time plus safety buffer stock:

$$\text{Calculated ROP} = \lceil \left(\text{ADU} \times \text{Lead Time Days} \right) + \text{Safety Stock} \rceil$$
$$\text{Effective ROP} = \max\left(\text{Manual ReorderPt}, \text{Calculated ROP} \right)$$

3-Tier Stock Alert Classification

Condition	Status	Operational Action
`stock ≤ effectiveRop`	**Critical**	Reorder immediately; stock is at or below replenishment threshold
`effectiveRop < stock < [1.5 × effectiveRop]`	**Warning**	Low buffer remaining; prepare next purchase order
`stock ≥ [1.5 × effectiveRop]`	**Normal**	Healthy inventory coverage

Immutable Double-Entry Stock Ledger

Every inventory mutation (`SALE` , `RESTOCK` , `ADJUSTMENT` , `CANCEL_RESTORE`) runs inside a PostgreSQL transaction and records `previousBalance` , `quantityChange` , and `newBalance` in `stock_movements` .

3. Concurrency, Deadlock Prevention & Idempotency

Source: `salesService.ts`, `orderService.ts`, `idempotency.ts`, `accountCleanupService.ts`, `unpaidOrderCleanupService.ts`

Canonical Lock Ordering (Deadlock Prevention)

- **Checkout (POS & Online):** Cart items are deduplicated into a map and sorted in **ascending `productId` order** before acquiring pessimistic `SELECT ... FOR UPDATE` row locks in PostgreSQL.
- **Order State Transitions / Cancellation:** Acquires `SalesOrder FOR UPDATE` first, followed by `products FOR UPDATE` in ascending `productId` order. Consistent lock acquisition ordering prevents circular-wait deadlocks under concurrent requests.

SHA-256 Request Idempotency

- Requests carrying `request_key` or `Idempotency-Key` (16–80 chars) have their JSON body hashed via **SHA-256** and stored in `idempotency_logs` keyed by `(userId, requestKey)`.
- **Exact retry:** Replays the saved HTTP status and JSON response without re-charging or double-deducting stock.
- **Mutated payload reuse:** Returns `409 Conflict` (`Changed payload cannot reuse key`).

PostgreSQL Advisory Locks (Cluster Coordination)

Background jobs guard against duplicate execution across PM2 cluster workers using `pg_try_advisory_lock(...)` alongside primary-worker checks (`NODE_APP_INSTANCE === '0'`):

- **Unverified Account Cleanup (Daily 3:00 AM PHT):** Advisory lock ID `8492019`
- **Unpaid E-Wallet Auto-Cancellation (5-minute poll):** Advisory lock ID `8492027`

4. Security, Validation & Throttling

Source: `backend/src/middleware/security.ts`, `backend/src/routes/orderRoutes.ts`, `backend/src/services/orderService.ts`

Mechanism	Implementation	Purpose
Magic-Byte Verification	Inspects binary header bytes: JPEG (`0xFF 0xD8 0xFF`) & PNG (`0x89 0x50 0x4E 0x47`), max 5 MB	Prevents spoofed file extensions, SVG/HTML XSS payloads, and double-extension uploads
Window Rate Limiting	`express-rate-limit`: `apiLimiter` (300 req / 15m) & `customerCancelLimiter` (keyed per user ID)	Protects endpoints against brute-force bursts and cancellation spam
Input Sanitization & CORS	`sanitize-html` (strips all tags/attributes) + exact-origin `CORS_ORIGINS` whitelist	Prevents stored XSS and unauthorized cross-origin requests
Server-Authoritative Pricing	Computes totals exclusively from locked DB prices; rejects stale `expected_total` (> 0.01 diff) with `409`	Prevents client-side price tampering and stale-cart charges